

Documentación Técnica: Dolce Vita Pastelería

Desarrollo de Software

26 de mayo de 2026

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
3. Backend: PasteleriaAPI	2
3.1. Tecnologías Utilizadas	2
3.2. Entidades de Dominio (Models/Entities)	2
3.3. Controladores (Endpoints)	3
4. Frontend: PasteleriaMaui	3
4.1. El Patrón MVVM	3
4.2. Componentes Principales (Views y ViewModels)	3
4.3. Servicios de Integración	4
5. Flujo de Trabajo Típico: Consultar Pasteles	4
6. Conclusión	4

1. Introducción

El proyecto **Dolce Vita Pastelería** es una solución de software integral para la gestión de una pastelería. Se compone de dos proyectos principales: un Backend desarrollado en ASP.NET Core (Web API) y un cliente Frontend multiplataforma desarrollado en .NET MAUI utilizando el patrón de diseño estricto Model-View-ViewModel (MVVM).

2. Arquitectura del Sistema

El sistema emplea una arquitectura *Cliente-Servidor* donde:

- **Servidor (Backend - PasteleriaAPI):** Se encarga de la lógica de negocio, acceso a datos y exposición de servicios a través de una API RESTful. Utiliza Entity Framework Core para la persistencia de datos en SQL Server.
- **Cliente (Frontend - PasteleriaMaui):** Consume los servicios de la API. Está estructurado utilizando el patrón MVVM, asegurando un bajo acoplamiento entre la lógica de presentación (ViewModels) y la interfaz de usuario (Views).

3. Backend: PasteleriaAPI

El backend está construido sobre .NET 10 empleando C#. Actúa como el punto central de verdad, gestionando entidades como Pasteles, Pedidos, Usuarios y Catálogos dinámicos.

3.1. Tecnologías Utilizadas

- ASP.NET Core Web API
- Entity Framework Core
- SQL Server LocalDB
- OpenAPI / Swagger

3.2. Entidades de Dominio (Models/Entities)

Las entidades principales del sistema incluyen:

- **Pastel:** Representa los productos a la venta, incluyendo información como nombre, descripción, precio y detalles de sus componentes (bizcocho, relleno, glaseado).
- **Pedido:** Almacena los pedidos realizados por los clientes, relacionando al usuario, los pasteles comprados y el estado del pedido (Pendiente, Entregado, etc.).
- **Merma:** Entidad para llevar el registro de productos en mal estado o no vendidos.
- **Usuario:** Manejo de roles (Ej. Administrador, Cliente) y credenciales.
- **Catálogos:** Conjunto de entidades (CatBizcocho, CatRelleno, CatGlaseado, CatCategoria) para normalizar y poblar dinámicamente los selectores de la aplicación.

3.3. Controladores (Endpoints)

Los controladores procesan las peticiones HTTP (GET, POST, PUT, DELETE):

- **PastelController:** Gestiona las operaciones CRUD para pasteles y catálogos. Se apoya en `IPastelService` para desacoplar la lógica directa de base de datos.
- **PedidoController:** Maneja la creación, lectura y actualización de estado de los pedidos y el historial.
- **MermaController:** Permite registrar y obtener reportes de mermas.
- **AuthController:** Endpoint para la autenticación de usuarios y registro de nuevas cuentas.

4. Frontend: PasteleriaMaui

Aplicación multiplataforma (Windows, Android, iOS, MacCatalyst) construida con .NET MAUI.

4.1. El Patrón MVVM

La aplicación utiliza **MVVM** de manera estricta:

- **Models:** Representan los datos transferidos por la API (Data Transfer Objects - DTOs).
- **Views (XAML):** Interfaz gráfica con la que interactúa el usuario. No contienen lógica de negocio en su código base (*Code-Behind*).
- **ViewModels:** Contienen la lógica de presentación, implementan la interfaz `INotifyPropertyChanged` y enlazan datos (Data Binding) a las vistas. Exponen comandos (`ICommand`) para acciones como botones.

4.2. Componentes Principales (Views y ViewModels)

- **Inicio / Login / Register:** Pantallas de bienvenida y acceso al sistema mediante el control de la sesión con la clase `AppSession`.
- **ListaPastelesPage & ListaPastelesViewModel:** Consulta a la API para obtener la lista de pasteles. Utiliza una `ObservableCollection<Pastel>` para actualizar automáticamente la interfaz. Si el usuario es administrador (`IsAdmin`), habilita opciones de edición y creación.
- **DetallePastelPage:** Permite ver los detalles específicos de un pastel seleccionado mediante paso de parámetros en el `Shell.Current.GoToAsync`.
- **RegistrarPedidoPage & PedidoPersonalizadoPage:** Flujos para que los clientes levanten un nuevo pedido. Utilizan componentes tipo `Picker` que se llenan a partir de los catálogos del sistema para evitar errores de escritura.

- **CatalogosPage:** Interfaz exclusiva para administradores donde se pueden realizar altas, bajas y modificaciones directas de categorías, bizcochos, rellenos, etc.
- **Reportes (ReporteMermaPage, ReporteEntregasPage):** Pantallas gerenciales para obtener las métricas y reportes de la base de datos consultando a sus respectivos controladores.

4.3. Servicios de Integración

La carpeta **Services** en el proyecto MAUI (ej. **PastelApiService**) contiene las clases encargadas de realizar llamadas HTTP (usando **HttpClient**) hacia **PasteleriaAPI**. Estas clases serializan y deserializan los objetos JSON enviados y recibidos.

5. Flujo de Trabajo Típico: Consultar Pasteles

1. El usuario ingresa a **ListaPastelesPage**.
2. La vista se enlaza con **ListaPastelesViewModel**.
3. Al inicializarse, el **ViewModel** ejecuta el comando **CargarPastelesCommand**.
4. El comando llama al método asíncrono en **PastelApiService.GetPastelesAsync()**.
5. El servicio realiza una petición **GET** a **/api/Pastel**.
6. El **PastelController** recibe la petición, consulta la base de datos vía **IPastelService** (Entity Framework) y devuelve un arreglo JSON.
7. **PastelApiService** deserializa la respuesta y la envía al **ViewModel**.
8. El **ViewModel** actualiza su **ObservableCollection**, lo que provoca que la **View** renderice la lista actualizada en la pantalla.

6. Conclusión

Dolce Vita Pastelería sigue buenas prácticas de ingeniería de software moderno: una API escalable que funciona como Single Point of Truth (SPoT) e interfaces de usuario robustas manejadas por Binding y MVVM. El código está modularizado para permitir el crecimiento futuro, como añadir pasarelas de pago o expandirse a nuevas plataformas.